TEXT CLASSIFICATION USING NAIVE BAYES

Python NLP Laboratory — Step-by-step execution and explanation

Objective: Develop a Naive Bayes classifier to classify text documents, such as spam/ham emails or positive/negative reviews. This experiment implements Spam/Ham email classification.

1. What is Naive Bayes?

Naive Bayes is a supervised Machine Learning classification algorithm based on Bayes' theorem. It calculates the probability that a document belongs to a particular category based on the words present in that document.

Example: “Congratulations! You have won a lottery. Claim your prize now.” is expected to be Spam. “Dear student, your examination timetable is available.” is expected to be Ham (a genuine email). The computer learns from labelled examples and classifies new emails.

2. Understanding Bayes' theorem

P(C|D) = [P(D|C) × P(C)] / P(D)

P(C|D) is the probability that document D belongs to category C; P(D|C) is the probability of observing the document given the category; P(C) is the prior probability of the category; and P(D) is the probability of observing the document.

For an email containing words such as “win”, “prize” and “money”, the classifier estimates the probability of Spam and the probability of Ham. If the Spam probability is higher, the email is classified as Spam.

3. How does Naive Bayes work?

The workflow is: input emails → text preprocessing → numerical feature extraction → Naive Bayes classifier → predicted category (Spam or Ham).

The classifier learns patterns from example emails. In this experiment, it learns which words tend to occur in Spam emails and which tend to occur in Ham emails.

4. What is Multinomial Naive Bayes?

Different Naive Bayes variants are suited to different kinds of input data:

Algorithm	Typical application
Gaussian Naive Bayes	Continuous numerical data
Multinomial Naive Bayes	Text classification and word frequencies
Bernoulli Naive Bayes	Binary features such as word present/absent

This experiment uses **Multinomial Naive Bayes** because the input is represented using word-frequency features.

5. Install the required Python library

Open Command Prompt or the VS Code terminal and execute:

```
pip install scikit-learn
```

The program uses CountVectorizer to convert words into numerical frequency vectors, train_test_split to divide the dataset, MultinomialNB for classification, and accuracy_score, classification_report and confusion_matrix for evaluation.

6. Complete Python program

Create a Python file named **naive_bayes_text_classification.py** and enter the following code:

```
# TEXT CLASSIFICATION USING NAIVE BAYES
```

```
# STEP 1: Import required libraries
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB

from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
from sklearn.metrics import confusion_matrix

# STEP 2: Create sample email documents
emails = [
    # SPAM EMAILS
    "Congratulations you have won a lottery prize",
    "Claim your free cash reward now",
    "Win money by clicking this link",
    "You have received a free gift voucher",
    "Get rich quickly with this investment",
    "Limited offer buy now and save money",
    "You are selected for a lucky draw",
    "Earn extra income from home",
    "Claim your prize before the offer expires",
    "Free entry in the cash prize contest",
    "Congratulations claim your reward today",
    "Exclusive deal get free shopping vouchers",

    # HAM EMAILS
    "Please attend the department meeting tomorrow",
    "Your examination timetable is available",
    "The project submission deadline is Friday",
    "Please send the assignment to your teacher",
    "The college library will remain open today",
    "Your class schedule has been updated",
    "The meeting is scheduled for Monday",
    "Please find the attached lecture notes",
    "The examination results will be announced soon",
    "Submit your project report before the deadline",
    "The faculty meeting starts at ten",
    "Students should attend the laboratory session"
]

# STEP 3: Define category labels
labels = [
    "Spam", "Spam", "Spam", "Spam",
    "Spam", "Spam", "Spam", "Spam",
    "Spam", "Spam", "Spam", "Spam",

    "Ham", "Ham", "Ham", "Ham",
    "Ham", "Ham", "Ham", "Ham",
    "Ham", "Ham", "Ham", "Ham"
]

# STEP 4: Divide data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    emails, labels,
    test_size=0.25,
    random_state=42,
    stratify=labels
)

# STEP 5: Convert text into numerical features
vectorizer = CountVectorizer(stop_words="english")
X_train_count = vectorizer.fit_transform(X_train)
X_test_count = vectorizer.transform(X_test)

print("VOCABULARY:")
print(vectorizer.get_feature_names_out())
print("\nTRAINING FEATURE MATRIX:")
print(X_train_count.toarray())

# STEP 6: Create Naive Bayes classifier
model = MultinomialNB()

# STEP 7: Train the classifier
model.fit(X_train_count, y_train)

# STEP 8: Predict email categories
y_pred = model.predict(X_test_count)

# STEP 9: Evaluate the classifier
accuracy = accuracy_score(y_test, y_pred)
print("\n===== NAIVE BAYES RESULTS =====")
print("Accuracy:", round(accuracy, 4))

# STEP 10: Display classification report
print("\nCLASSIFICATION REPORT:")
print(classification_report(
    y_test, y_pred,
    zero_division=0
))
```

```
# STEP 11: Display confusion matrix
print("\nCONFUSION MATRIX:")
print(confusion_matrix(
    y_test, y_pred,
    labels=["Spam", "Ham"]
))

# STEP 12: Test new emails
new_emails = [
    "Congratulations you won free money",
    "Please attend the college meeting tomorrow",
    "Claim your free prize now",
    "Submit your assignment before Friday"
]

# Convert new emails into numerical features
new_email_count = vectorizer.transform(new_emails)

# Predict categories
predictions = model.predict(new_email_count)

print("\n===== NEW EMAIL CLASSIFICATION =====")
for email, category in zip(new_emails, predictions):
    print("\nEmail:", email)
    print("Predicted Category:", category)

# STEP 13: Display prediction probabilities
probabilities = model.predict_proba(new_email_count)
print("\n===== CLASSIFICATION PROBABILITIES =====")
for email, probability in zip(new_emails, probabilities):
    print("\nEmail:", email)
    for category, value in zip(model.classes_, probability):
        print(category, ":", round(value, 4))
```

7. Explanation of the Python program

Step 1 — Import the libraries

CountVectorizer converts text documents into numerical representations based on word frequencies. MultinomialNB creates the Naive Bayes classifier. train_test_split divides the dataset. The metrics functions evaluate classification performance.

Step 2 — Create sample email documents

The program contains 24 emails: 12 Spam emails and 12 Ham emails. These examples teach the computer how the two categories differ.

Step 3 — Define category labels

Each email has a corresponding label: Spam or Ham. The labels tell the computer the correct category for each training example.

Step 4 — Divide the dataset

```
X_train, X_test, y_train, y_test = train_test_split(
    emails, labels,
    test_size=0.25,
    random_state=42,
    stratify=labels
)
```

X_train contains training emails; X_test contains testing emails; y_train contains training labels; y_test contains actual testing labels. test_size=0.25 reserves 25% for testing and 75% for training. With 24 emails, this gives 18 training emails and 6 testing emails. random_state=42 makes the split reproducible. stratify=labels maintains category proportions.

Step 5 — Convert text into numerical features

```
vectorizer = CountVectorizer(stop_words="english")
X_train_count = vectorizer.fit_transform(X_train)
X_test_count = vectorizer.transform(X_test)
```

CountVectorizer builds a vocabulary and represents each document using word counts. stop_words='english' removes common English words such as “the”, “is” and “a”. fit_transform() learns the vocabulary from the training documents and creates their vectors. transform() converts test documents using the same vocabulary.

Example of Bag-of-Words

Consider two emails: “Free money win prize” and “Please attend college meeting”. The vocabulary is attend, college, free, meeting, money, please, prize, win.

Word	Email 1	Email 2
attend	0	1
college	0	1
free	1	0
meeting	0	1
money	1	0
please	0	1
prize	1	0
win	1	0

Email 1 becomes [0, 0, 1, 0, 1, 0, 1, 1]. Email 2 becomes [1, 1, 0, 1, 0, 1, 0, 0]. These lists are numerical feature vectors.

Step 6 — Create the Naive Bayes classifier

```
model = MultinomialNB()
```

This creates a Multinomial Naive Bayes classifier. It uses word-frequency features to estimate the probability that an email belongs to Spam or Ham.

Step 7 — Train the classifier

```
model.fit(X_train_count, y_train)
```

The classifier learns which words are associated with Spam and which words are associated with Ham. For example, words such as “lottery”, “prize”, “free” and “claim” may be associated with Spam in this sample dataset. Words such as “meeting”, “assignment”, “examination” and “college” may be associated with Ham.

Step 8 — Predict email categories

```
y_pred = model.predict(X_test_count)
```

The trained model predicts whether each testing email is Spam or Ham.

Step 9 — Evaluate accuracy

```
accuracy = accuracy_score(y_test, y_pred)
```

Accuracy = (number of correct predictions / total predictions) × 100. For example, if 5 out of 6 emails are correctly classified, accuracy = (5 / 6) × 100 = 83.33%.

Step 10 — Classification report

```
print(classification_report(y_test, y_pred, zero_division=0))
```

The report displays precision, recall, F1-score and support for each category. Support is the number of actual test emails in that category.

Step 11 — Confusion matrix

```
print(confusion_matrix(
    y_test, y_pred,
    labels=["Spam", "Ham"]
))
```

The confusion matrix compares actual categories with predicted categories. Rows represent actual categories and columns represent predicted categories.

Actual / Predicted	Spam	Ham
Spam	Correct Spam	Spam classified as Ham
Ham	Ham classified as Spam	Correct Ham

Step 12 — Classify new emails

The program supplies four new emails that were not used during training. vectorizer.transform() converts them using the learned vocabulary, and model.predict() assigns a category to each email.

Step 13 — Display classification probabilities

```
probabilities = model.predict_proba(new_email_count)
```

predict_proba() returns the estimated probability for each category. For example, a hypothetical result might be Ham = 0.10 and Spam = 0.90. The model would predict Spam because its estimated probability is higher. Actual probabilities are calculated when the program runs.

8. Expected output

The program displays the vocabulary, training feature matrix, accuracy, classification report, confusion matrix, new email predictions and classification probabilities. Numerical values are calculated at runtime.

```
===== NAIVE BAYES RESULTS =====
Accuracy: [calculated value]

CLASSIFICATION REPORT:
              precision  recall  f1-score  support
Ham
Spam
accuracy
macro avg
weighted avg

CONFUSION MATRIX:
[[... ...]
 [... ...]]

===== NEW EMAIL CLASSIFICATION =====
Email: Congratulations you won free money
Predicted Category: Spam

Email: Please attend the college meeting tomorrow
Predicted Category: Ham
```

9. How to execute the program

1. Open IDLE, VS Code, PyCharm or another Python environment.
2. Install Scikit-learn using `pip install scikit-learn`.
3. Create a Python file named **naive_bayes_text_classification.py**.
4. Paste the complete program and save it.
5. Open a terminal in the file's folder and run:

```
python naive_bayes_text_classification.py
```

Only Scikit-learn is required. The sample email dataset is included in the script, so no external dataset download is needed.

10. Applications of Naive Bayes

• Email spam detection
• Sentiment analysis
• News classification
• Document categorization
• Fake news detection
• Customer feedback analysis
• Medical text classification

11. Difference between Naive Bayes and SVM

Naive Bayes	SVM
Based on Bayes' theorem.	Based on separating hyperplanes.

Naive Bayes	SVM
Uses probability calculations.	Uses support vectors and margins.
Uses a conditional independence assumption.	Does not require this assumption.
MultinomialNB works with word counts.	SVM works with high-dimensional text features.

12. Student exercises

1. Add 10 more Spam emails.
2. Add 10 more Ham emails.
3. Change the email classification into Positive and Negative movie review classification.
4. Try TF-IDF instead of CountVectorizer.
5. Compare Naive Bayes and SVM classification accuracy.
6. Enter five new emails and observe the predicted categories.

13. Quick knowledge check

Question 1: Which theorem is used by Naive Bayes?
Answer: Bayes' theorem.

Question 2: Which Naive Bayes algorithm is suitable for word-frequency features?
Answer: Multinomial Naive Bayes.

Question 3: What does CountVectorizer do?
Answer: Converts text into numerical word-frequency features.

Expected laboratory outcome: Students should be able to understand Bayes' theorem, convert text into numerical features, implement a Multinomial Naive Bayes classifier, classify unseen emails and evaluate model performance.

Teaching suggestion: First explain Bayes' theorem using a simple Spam/Ham example. Then demonstrate how CountVectorizer converts words into numbers. Finally, execute the program and explain how Naive Bayes predicts the category of a new email.

Note: The 24-email dataset is a small educational example. Its accuracy should not be interpreted as real-world spam detection performance. A practical spam detection system requires a much larger and more diverse dataset.